Memory-level parallelism in BSGS, inversion-free rho start-up

Study `research/generic_dlp_memory_parallelism_20261017` · run `run-20261017T0201Z` · 2026-10-17 · PR #2048

Class: engineering (AGENTS.md §3).

- Two changes to the single-word generic-DLP engines lower one-target wall time.
- Every counted output is identical between baseline and candidate: answers, steps, table entries, candidates, distinguished points, and the field operations per visited point.
- `S` and every IC/rho ratio are unchanged.
- Host: one x86-64 Xeon (Sapphire/Emerald Rapids class) at 2.1 GHz, a 4-vCPU KVM guest, isolation L0 with contended runs left out.

1. What changed

1. **`bsgs_fast`: table slots are prefetched a round before they are probed.**
   - The baby-step table outgrows L2 past about `2^36`.
   - At load factor ½ the probe's "empty?" branch depends on the missing load, so successive misses serialise.
   - The kernels now hint each visited point's `x` to the table before the visit. For `paired` and `co-Z`, a round is computed and hinted in one pass and visited in a second, in the original order.
   - Visits, early exit, charged steps and field operations are unchanged; a test pins all four.
2. **`prime_fast`: Jacobian scalar multiplication, Straus–Shamir `combine`.**
   - `rho_parallel` builds its 1024-entry `r`-adding table and every walk start with `combine(g, q, u, v)`. That was two affine double-and-add ladders paying a binary-GCD inversion per group operation.
   - None of this work is in `total_steps`, and all of it is target-dependent.
   - It now costs two inversions per call. The results equal the affine ladder's on every point of four small curves and on the ladder curves.

Tried and rejected. `MADV_HUGEPAGE` on the table.

- In exploratory runs it sped up the search phase but made the build phase erratic: THP defrag is `madvise` here, so the first touch compacts synchronously.
- It was not shipped.

2. Method

- **Arms.** A is `main` at `b1bf5b893`; A′ is the same binary, as the A/A control; B is commit `c699775dd`.
- **Rounds.** 7 rotated rounds, 5 at 52 bits.
- **Workload.** One target per process, a cold full-group solve.
- **Isolation.** Every run is under `tools/isolated_bench.py`.
- **Rule.** "improves: every per-round A/B ratio above the largest per-round A/A2 ratio (and above 1); regresses: every one below the smallest (and below 1); otherwise no measurable change".
- **Contention.** 26 of 438 timed runs were contended and are left out, as are 0 of 30 PARI runs.
- **Pinned outputs.** Identical in every configuration: yes.
- **Field-operation rows.** 120 rows compared, all identical.
- **Curves.** They are in `inputs/curves.json`, with `#E` checked by PARI/GP:
  - BSGS uses $y^2 = x^3 + 7$ (W11): `icv1-fp40-t1502959-42c6aada`, `icv1-fp44-tm1269095-f56aca35`, `icv1-fp48-t14984575-af5d5615`, `icv1-fp52-tm131765273-051185e0`;
  - rho uses $a = -3$: `icv1-fp32-t7319-90da43e9`, `icv1-fp36-t454193-44a20818`, `icv1-fp40-tm924137-0ecaaeb9`, `icv1-fp44-tm6431881-43acc95d`, `icv1-fp48-tm15186475-c683ebfc`, `icv1-fp52-t113543683-af85fb1e`.

3. Results

3.1. BSGS, one target, full group, negation map

Times are in seconds; ratios are A/B. The per-round ranges are min–max.

threads	bits	kernel	A `total_s`	B `total_s`	A/B	A/B per round	verdict
1	40	chain	0.109	0.0858	1.27	[0.959, 1.492]	no measurable change
1	40	paired	0.0565	0.0417	1.354	[1.026, 1.54]	no measurable change
1	44	chain	0.3482	0.2415	1.442	[1.287, 1.617]	improves
1	44	paired	0.2265	0.1142	1.984	[1.32, 2.652]	improves
1	48	chain	1.4351	1.1003	1.304	[1.243, 1.401]	improves
1	48	paired	1.0429	0.5543	1.881	[1.739, 1.957]	improves
1	52	chain	6.5163	4.9373	1.32	[1.241, 1.402]	improves
1	52	paired	4.6114	2.6462	1.743	[1.667, 1.847]	improves

3	48	chain	0.581	0.435	1.336	[1.232, 1.44]	improves
3	48	paired	0.4152	0.2767	1.501	[1.386, 1.853]	improves

The giant (search) phase moved by about as much as the build. The frozen targets have small logarithms, so `total_s` is mostly the table build; giant steps are pinned identical, so a `search_s` ratio is a per-step ratio. The search ratios, from the run report:

- paired: 1.885 at 44 bits, 2 at 48, 1.667 at 52;
- chain: 1.391 at 48 bits.

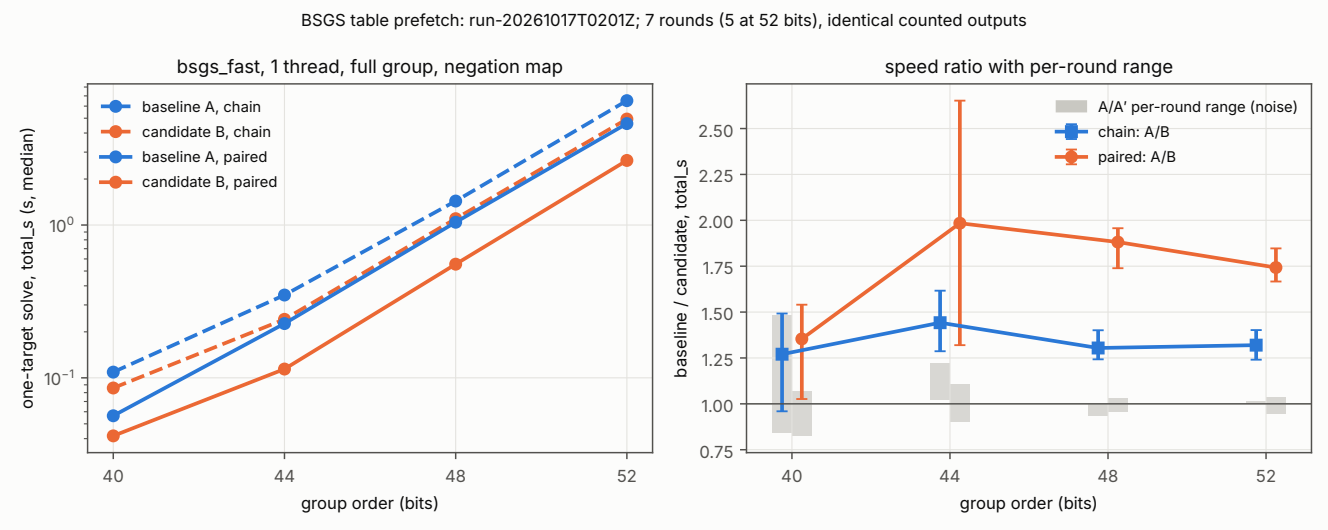


Figure 1: BSGS medians and A/B with per-round ranges against the A/A′ noise band (run-20261017T0201Z).

3.2. Rho (`prime_fast::rho_parallel`), one target, negation map

threads	bits	A `solve_s`	B `solve_s`	A/B	A/B per round	verdict	start-up alone, A/B
1	32	0.0312	0.0053	5.836	[3.577, 7.177]	improves	6.857
1	36	0.0532	0.0142	3.745	[2.505, 3.912]	improves	7.961
1	40	0.1071	0.0588	1.822	[1.448, 2.075]	improves	9.105
1	44	0.4518	0.3894	1.16	[0.979, 1.457]	no measurable change	10.959
1	48	1.1729	1.0332	1.135	[1.07, 1.27]	improves	13.023
1	52	1.7896	1.648	1.086	[0.902, 1.151]	no measurable change	16.094
3	32	0.0283	0.0109	2.595	[1.978, 4.169]	improves	6.774
3	36	0.0714	0.0364	1.961	[1.627, 2.097]	improves	8.029
3	40	0.0728	0.0304	2.393	[1.997, 3.793]	improves	8.5
3	44	0.1612	0.0928	1.737	[1.496, 2.166]	improves	10.996
3	48	0.2142	0.1233	1.737	[1.546, 2.195]	improves	13.506
3	52	1.2046	1.0852	1.11	[1.014, 1.144]	no measurable change	14.783

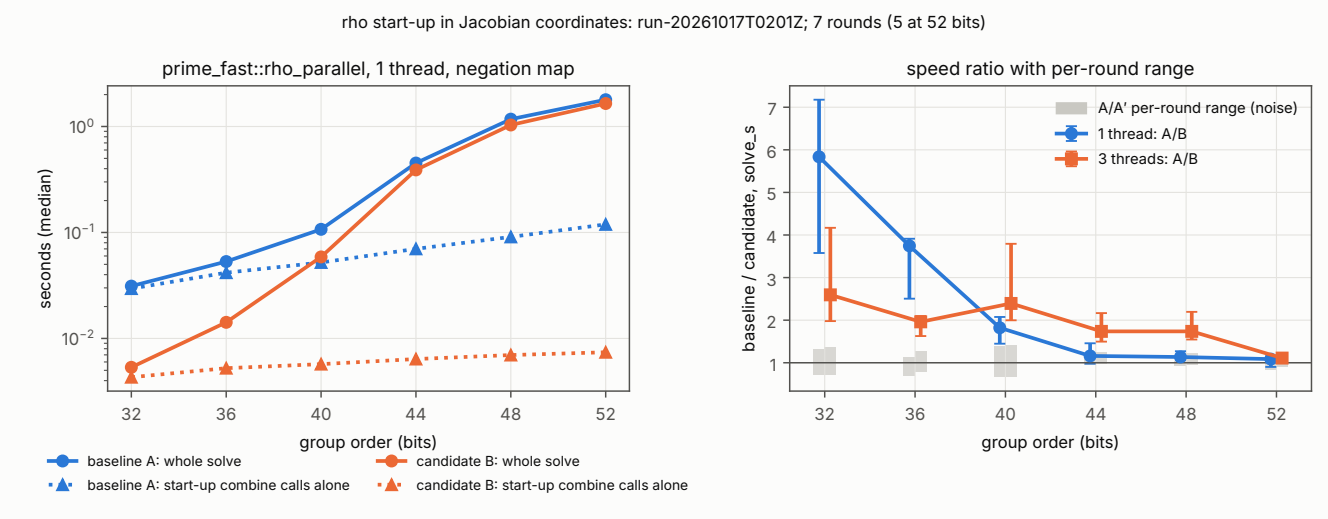


Figure 2: Rho solve and start-up medians, and A/B at 1 and 3 threads (run-20261017T0201Z).

3.3. External reference: PARI/GP `elllog` (AGENTS.md §15)

PARI/GP 2.15.4, single-threaded, on the same curves and the same targets, timed by `getabstime()` around `elllog`. Every answer was verified. Which generic method PARI ran is not observable: its help text and debug domains print none.

curves	bits	PARI median (s)	A median (s)	B median (s)	B / PARI	A / PARI
bsgs	40	1.006	0.0565	0.0417	0.0415	0.0561
bsgs	44	4.462	0.2265	0.1142	0.0256	0.0508
bsgs	48	18.495	1.0429	0.5543	0.03	0.0564
bsgs	52	27.749	4.6114	2.6462	0.0954	0.1662
rho	32	0.032	0.0312	0.0053	0.1669	0.974
rho	36	0.141	0.0532	0.0142	0.1007	0.377
rho	40	0.679	0.1071	0.0588	0.0866	0.1577
rho	44	3.162	0.4518	0.3894	0.1231	0.1429
rho	48	11.754	1.1729	1.0332	0.0879	0.0998
rho	52	80.788	1.7896	1.648	0.0204	0.0222

Growth is $\log_2$ of the median-time ratio per bit; $\sqrt{n}$ scaling is 0.5:

- bsgs curves, 40⟶52 bits: PARI 0.399, A 0.529, B 0.499.
- rho curves, 32⟶52 bits: PARI 0.565, A 0.292, B 0.413.

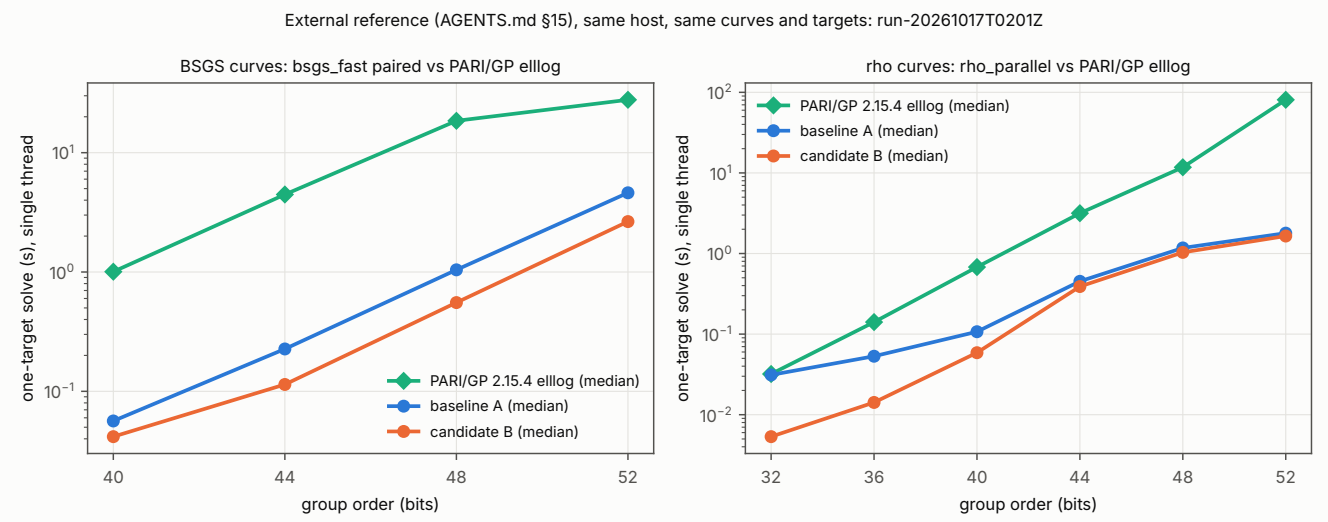


Figure 3: PARI/GP `elllog` against both native arms on the same curves and targets.

Scope of the external comparison.

- At every size measured, both native arms are faster than PARI/GP on this host. Over this range the native rho grows more slowly than $\sqrt{n}$, because its fixed start-up is amortised, so the 52-bit gap is not a constant to extrapolate.
- The fastest published ECDLP implementations (record-scale distinguished-point rho on GPUs and FPGAs, at 100 bits and more) were not reproduced.
- Sage is not installed, and Magma needs a licence that this host lacks.
- No "fastest" claim is made.

4. Interpretation (labelled)

- **BSGS was bound by memory latency, not arithmetic.** Taking the probe's miss off the dependency chain saved 23–31% (chain) and 43–50% (paired) of one-target time at 44–52 bits.
  - Paired gained more: its arithmetic is 3.8–4.5× smaller (3.84–3.94 against 14.6–17.9 M-equivalents per point, from this run's op counts), so it was the more memory-bound kernel.
  - At three threads, which share one table and one memory system, the gain is smaller.
- **Rho at small sizes was dominated by its uncounted start-up.**
  - That start-up is now 6.8–16.1× cheaper.
  - The solve gain shrinks as the $\sqrt{n}$ walk grows. At 44 and 52 bits (one thread) it is within noise.

5. Limits

- No change to S, to a counted cost, or to any IC/rho comparison. No method ranking; the §5a band does not apply.
- One host class only. Arm64, GPUs, other x86 parts and bare metal were not measured.
- One frozen target per curve, as the protocol fixes; the targets have small logarithms. The 40-bit BSGS rows are no measurable change by the rule.

6. Audit and next steps

The audit behind this study covers BSGS, rho, kangaroo, Gaudry–Schost, Pohlig–Hellman and the `ecc/` stacks. It found 26 findings in all, with how each is known and its status, in `AUDIT_FINDINGS.md`.

- **Fixed in a separate PR (pending):**
  - the `pollard_collab` negation-map 2-cycle escape, which never fired;
  - Pohlig–Hellman's `O(q)` digits, unbounded factorisation and unchecked answer.
- **Open:**
  - a coarse scalar bit-length timing leak in `Point::scalar_mul_ct`'s generic fallback;
  - Gaudry–Schost and collab branch bits overlapping the DP bits;
  - grumpy giants not following Bernstein–Lange;
  - `aut_folded_rho` not folding;
  - several reference-arm cost levers, each needing its own calibration.
- **Next engineering steps, named in the README:**
  - interleaved negation-map BSGS for one target (≈ `0.943`√n against `1.00`√n, derived);
  - a barrier-free single-worker baby phase;
  - coefficient-free rho steps;
  - a Jacobian `bsgs_fast` scalar multiplication.